

W4 Question Bank - Web and Flask Fundamentals

Web Fundamentals (8 Questions)

1. What is the difference between HTTP and HTTPS?
 2. Explain the HTTP request-response cycle with an example.
 3. What are the main HTTP methods and their purposes? (GET, POST, PUT, DELETE)
 4. What is the significance of HTTP status codes? Give examples of 2xx, 3xx, 4xx, and 5xx codes.
 5. What are HTTP headers? Give 3 examples and explain their use.
 6. What is CGI? How does it relate to web servers and applications?
 7. Compare monolithic vs microservices web application architecture.
 8. What is REST? How does it differ from SOAP?
-

Flask Basics (5 Questions)

9. What is Flask? Why is it called a micro-framework?
 10. How do you install Flask and set up a basic "Hello World" app?
 11. What is the role of `app = Flask(__name__)` in a Flask application?
 12. How do you run a Flask app in debug mode?
 13. What is JSON? How is it used in web APIs?
-

Flask Routing (8 Questions)

14. How do you define a route in Flask? Write a simple example.
 15. What is the difference between `@app.route('/')` and `@app.route('/home')`?
 16. How do you handle dynamic URL patterns (e.g., `/user/<username>`)?
 17. Explain the difference between `GET` and `POST` routes with examples.
 18. What are `request` and `response` objects in Flask?
 19. How do you access form data sent via POST in Flask?
 20. How do you return JSON from a Flask route?
 21. What is the purpose of `url_for()` in Flask?
-

HTTP Handling (7 Questions)

22. How do you handle different HTTP methods on the same route in Flask?
23. How do you return custom HTTP status codes (e.g., 201, 400) in Flask?
24. What is the difference between `abort(404)` and `return "Not Found", 404`?
25. How do you handle file uploads in Flask?
26. What is `request.args`, `request.form`, and `request.files`?

- 27. How do you set cookies in a Flask response?
 - 28. How do you read cookies from a request in Flask?
-

Advanced Flask Features (10 Questions)

- 29. What are Flask templates? How do you render one?
 - 30. Explain Jinja2 templating syntax with examples (`{{ }}`, `{% %}`).
 - 31. How do you pass variables from a route to a template?
 - 32. What are template inheritance and blocks in Flask?
 - 33. What are Flask configuration files? How do you load `config.py`?
 - 34. Explain development vs production environments in Flask.
 - 35. What are Flask Blueprints? When and why should you use them?
 - 36. How do you register a Blueprint in the main app?
 - 37. What is middleware in Flask? How do you implement a simple one?
 - 38. What are `before_request` and `after_request` decorators?
-

Security & Best Practices (8 Questions)

- 39. What is JWT? How is it used for authentication?
 - 40. How do you implement JWT login and token verification in Flask?
 - 41. What is session management in Flask? How are sessions stored by default?
 - 42. How do you make sessions secure (e.g., HTTPS, secure cookies)?
 - 43. What is CORS? How do you enable it in Flask?
 - 44. How do you prevent CSRF attacks in Flask forms?
 - 45. What are common security vulnerabilities in Flask apps? (XSS, SQLi, etc.)
 - 46. How do you validate and sanitize user input in Flask?
-

API Development (6 Questions)

- 47. What are REST API design principles? (stateless, resource-based, etc.)
 - 48. How do you serialize Python objects to JSON in Flask?
 - 49. How do you deserialize JSON data from a POST request?
 - 50. How do you test a Flask API using Postman?
 - 51. How do you document a Flask REST API (e.g., using Flask-RESTx or Swagger)?
 - 52. What is the purpose of `Content-Type` and `Accept` headers in APIs?
-

Database Integration – Python + SQL (12 Questions)

- 53. How do you connect to a SQLite database in Python using `sqlite3`?

- 54. How do you connect to MySQL/PostgreSQL using Python drivers?
 - 55. What is the difference between `sqlite3` and `PyMySQL/psycopg2`?
 - 56. Write a Python script to perform basic CRUD operations using `sqlite3`.
 - 57. What is a Data Access Object (DAO) pattern? Give an example.
 - 58. How do you prevent SQL injection in raw Python SQL queries?
 - 59. What is connection pooling? Why is it important?
 - 60. How do you handle database transactions in Python (commit/rollback)?
 - 61. What is the difference between `fetchone()`, `fetchall()`, and `fetchmany()`?
 - 62. How do you execute parameterized queries in Python?
 - 63. What are ORM vs raw SQL? List pros and cons.
 - 64. How do you log database errors in a Python application?
-

Complete Integration – Flask + Database (16 Questions)

- 65. How do you integrate SQLAlchemy with Flask?
- 66. What is Flask-SQLAlchemy? How do you initialize it?
- 67. How do you define a database model using SQLAlchemy?
- 68. Explain one-to-many and many-to-many relationships with examples.
- 69. How do you perform database migrations using Flask-Migrate?
- 70. How do you seed initial data into a database using Flask?
- 71. How do you build a complete CRUD API using Flask + SQLAlchemy?
- 72. How do you query related models (e.g., JOINS) in SQLAlchemy?
- 73. What is the `session` object in SQLAlchemy? When is it committed?
- 74. How do you handle database connection errors in Flask?
- 75. How do you implement pagination in a Flask API with SQLAlchemy?
- 76. How do you integrate JWT authentication with database user lookup?
- 77. How do you write integration tests for Flask + SQLAlchemy routes?
- 78. How do you use `db.create_all()` vs migrations in development?
- 79. How do you roll back a failed transaction in Flask-SQLAlchemy?
- 80. How do you document a full-stack Flask + SQLAlchemy API?